Meowcessor CPU Project

Register File & ALU Datapath Integration Design Document

Aadi

September 17, 2026

Contents

1 Introduction **2**

2 What Is a Register File? **2**
2.1 The Memory Hierarchy Analogy 2
2.2 How a Single Register Holds a Value 2

3 Architecture **3**
3.1 Specification Summary 3
3.2 Port Interface 4
3.3 Block Diagram 4

4 Design Details **4**
4.1 Hardwired Zero Register (x_0) 4
4.2 Asynchronous (Combinational) Read Ports 5
4.3 Synchronous Write Port 5
4.4 Internal Write-Through Bypass (Forwarding) 5
4.4.1 The Problem 5
4.4.2 The Solution 5

5 ALU Datapath Integration **6**
5.1 Execution Sequence for a Single-Cycle Operation 6
5.2 Execution Sequence for a Multi-Cycle Operation 6
5.3 Chained Dependency Demonstration 7

6 Synthesis Results **7**

7 Verification Summary **8**

8 Conclusion **8**

1 Introduction

This document details the architecture, implementation, and verification of the **Register File** for the Meowcessor RISC-V 32-bit CPU, as well as its integration with the previously completed ALU wrapper. The register file is the CPU's fastest and most frequently accessed storage, providing operands to the ALU and receiving computation results on every instruction cycle.

The RISC-V RV32I base integer instruction set specifies 32 general-purpose registers, each 32 bits wide, with register `x0` permanently hardwired to zero. Our implementation adheres to this specification exactly.

2 What Is a Register File?

2.1 The Memory Hierarchy Analogy

A CPU interacts with multiple levels of storage, each trading capacity for speed:

1. **Registers** — The fastest storage. A tiny scratchpad (32 slots in RISC-V) built directly inside the CPU core from flip-flops. Access is effectively instantaneous (combinational read, single-cycle write).
2. **Cache (L1/L2/L3)** — Small but fast SRAM close to the CPU. Access takes a few cycles.
3. **Main Memory (RAM)** — Large but slower DRAM. Access takes tens to hundreds of cycles.
4. **Storage (SSD/HDD)** — Massive capacity but very slow. Access takes millions of cycles.

The register file sits at the very top of this hierarchy. Every arithmetic, logic, and data-movement instruction reads its operands from registers and writes its result back to a register.

2.2 How a Single Register Holds a Value

At the transistor level, a register is built from **D flip-flops** — circuits that capture and hold a single bit of data:

1. Two cross-coupled inverters form a **stable feedback loop**. Once a bit enters the loop, it reinforces itself indefinitely — this is the fundamental mechanism of digital memory.
2. A **clock-gated input** (the "door") controls *when* new data can enter the loop. The door only opens on the **rising edge** of the clock signal and is shut at all other times.
3. A **write-enable** signal acts as a second lock. Even if the clock edge arrives, the flip-flop only accepts new data when write-enable is asserted.

One 32-bit register is simply **32 D flip-flops side by side**, all sharing the same clock and write-enable. On a rising clock edge with write-enable high, all 32 flip-flops simultaneously swallow their respective input bit and hold it until the next enabled clock edge.

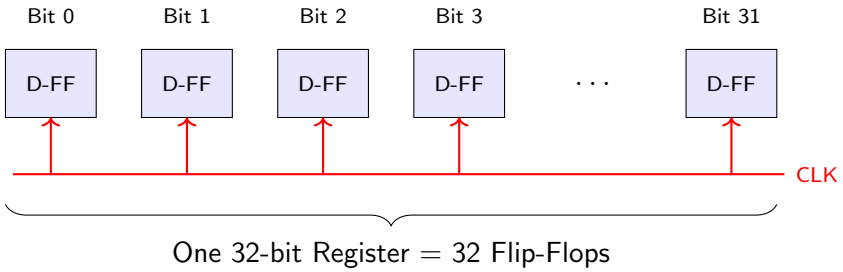


Figure 1: Structure of a single 32-bit register

3 Architecture

3.1 Specification Summary

Parameter	**Value**
Register Count	32 (x_0 through x_{31})
Register Width	32 bits (parameterised as `WIDTH`)
Address Width	5 bits ($\lceil \log_2 32 \rceil = 5$)
Read Ports	2 (asynchronous / combinational)
Write Port	1 (synchronous, rising edge of `clk`)
Reset	Active-low asynchronous (`rst_n`)
x_0 Behaviour	Hardwired to `32'h0000_0000`; writes ignored
Write-Through Bypass	Enabled (same-cycle forwarding)
Total Flip-Flops	$32 \times 32 = 1{,}024$

Table 1: Register File Specifications

3.2 Port Interface

Signal	Direction	Width	Description
`clk`	Input	1	System clock
`rst_n`	Input	1	Active-low asynchronous reset
`we`	Input	1	Write enable (active high)
`waddr`	Input	5	Write address (destination register rd)
`wdata`	Input	32	Write data
`raddr1`	Input	5	Read address for port 1 ($rs1$)
`rdata1`	Output	32	Read data from port 1
`raddr2`	Input	5	Read address for port 2 ($rs2$)
`rdata2`	Output	32	Read data from port 2

Table 2: Register File Port Map

3.3 Block Diagram

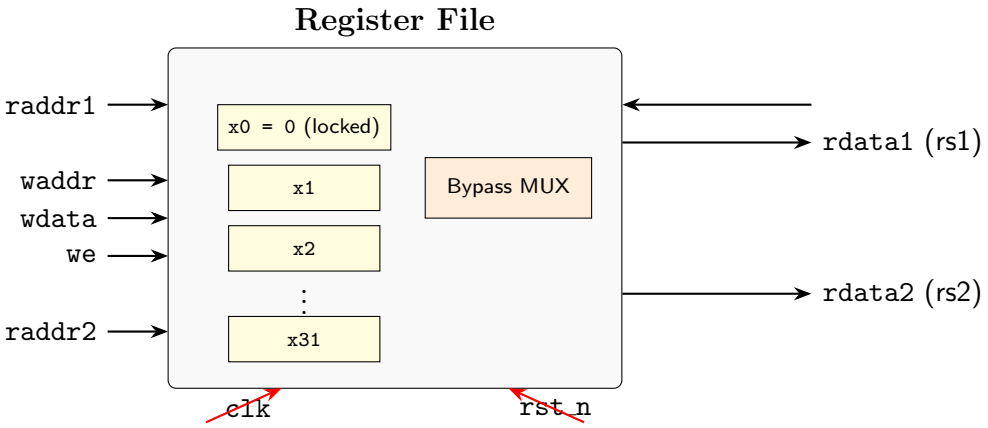


Figure 2: Register File Block Diagram

4 Design Details

4.1 Hardwired Zero Register (x_0)

The RISC-V specification mandates that register x_0 is permanently zero. Our implementation enforces this at two levels:

1. **Write Protection:** The synchronous write block checks `waddr != 0` before allowing any write. If `waddr == 0`, the write is silently discarded regardless of `we`.
2. **Read Clamping:** The combinational read logic checks `raddr == 0` and forces the output to `32'h0` without even accessing the storage array.

```
if (we && (waddr != {ADDR_WIDTH{1'b0}})) begin
    regs[waddr] <= wdata;
end
```

Listing 1: Write protection for x_0

This seemingly simple feature is architecturally powerful. It enables the ISA to use x_0 as a universal "discard register" (throw away unwanted results), a constant zero source (for moves, clears, and comparisons), and a NOP generator (`addi x0, x0, 0`).

4.2 Asynchronous (Combinational) Read Ports

Both read ports are implemented as purely combinational logic using continuous assignments. There is **no clock latency** for reads — the output data appears on `rdata1` / `rdata2` within the propagation delay of a multiplexer after `raddr1` / `raddr2` changes.

This is essential for single-cycle and pipelined CPU architectures where the decode/execute stage must receive operand values within the same clock cycle that the instruction is decoded.

4.3 Synchronous Write Port

Writes are captured on the **rising edge of** `clk` when `we == 1` and `waddr != 0`. This ensures that register contents change at well-defined, predictable instants — preventing glitches and metastability.

4.4 Internal Write-Through Bypass (Forwarding)

4.4.1 The Problem

Consider two back-to-back instructions:

```
Instruction 1:   add  x4, x1, x2     // x4 = x1 + x2 = 52
Instruction 2:   mul  x5, x4, x3     // x5 = x4 * x3
```

Instruction 2 needs the value of `x4` that was *just computed* by Instruction 1. However, the write to `x4` only takes effect on the rising clock edge. If Instruction 2 reads `x4` during the same cycle that Instruction 1 is writing it, it would read the *old, stale* value.

4.4.2 The Solution

The register file detects this conflict combinationally and **bypasses** the storage array, forwarding the write data directly to the read output:

```
1 assign rdata1 = (!rst_n || (raddr1 == {ADDR_WIDTH{1'b0}}))
2                     ? {WIDTH{1'b0}} :            // Reset or x0
3                   (we && (waddr == raddr1))
4                     ? wdata :                     // Bypass: forward
    fresh data
5                       regs[raddr1];               // Normal: read from
    storage
```

Listing 2: Write-through bypass logic

The priority chain is:

1. If in reset or reading x_0: output zero.
2. If the read address matches the current write address and write is enabled: output the write data directly (bypass).

3. Otherwise: output the stored register value normally.

This adds zero latency and eliminates the need for external forwarding logic or pipeline stalls for this class of data hazard.

5 ALU Datapath Integration

The register file connects to the ALU to form the core execution datapath of the Meowcessor. The data flow for an R-type instruction (e.g., `add x4, x1, x2`) proceeds as follows:

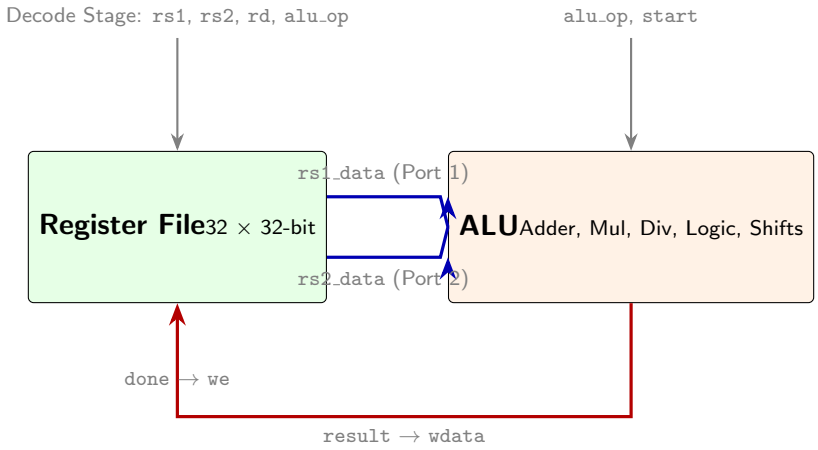


Figure 3: Register File ⟷ ALU Datapath

5.1 Execution Sequence for a Single-Cycle Operation

1. **Operand Fetch:** The decode stage drives `raddr1 = rs1` and `raddr2 = rs2`. The register file's combinational read ports immediately output the operand values on `rdata1` and `rdata2`.

2. **ALU Execution:** The ALU receives the operands, performs the operation selected by `alu_op`, and produces a result within the same clock cycle (for single-cycle ops like ADD, SUB, AND, etc.).

3. **Write-Back:** The ALU asserts `done`, which gates the register file's write enable. The result is written into `regs[rd]` on the next rising clock edge.

5.2 Execution Sequence for a Multi-Cycle Operation

For operations like MUL (17 cycles) or DIV (∼18 cycles):

1. Steps 1 is the same as above.

2. The ALU asserts `busy` and begins iterating internally.

3. The register file holds the `waddr = rd` value stable while waiting.

4. When the ALU finally asserts `done`, the result is written back to `regs[rd]` on the next rising clock edge.

5.3 Chained Dependency Demonstration

The integration testbench verifies a multi-instruction dependency chain that exercises the full datapath loop repeatedly:

$$
\begin{aligned}
x_{26} &= x_{21} + x_{22} = 5 + 15 = 20 \\
x_{27} &= x_{26} \times x_{23} = 20 \times 8 = 160 \quad \text{(uses result of previous line)} \\
x_{28} &= x_{27} - x_{24} = 160 - 10 = 150 \quad \text{(uses result of previous line)} \\
x_{29} &= x_{28} \div x_{25} = 150 \div 5 = 30 \quad \text{(uses result of previous line)}
\end{aligned}
$$

Each instruction depends on the result of the instruction before it. All four instructions passed verification, confirming that the register file correctly stores and supplies intermediate values across sequential ALU operations.

6 Synthesis Results

The register file was synthesised using Yosys to verify correct elaboration and resource usage.

Cell Type	Count	Purpose
`$_DFFE_PNOP_`	1,024	D flip-flops (32 regs × 32 bits)
`$_AND_`	2,083	Write decode & bypass logic
`$_OR_`	2,224	Read mux & bypass logic
`$_MUX_`	192	Read port multiplexers
`$_NOT_`	114	Inverters
`$_XOR_`	10	Address comparison
Total	**5,647**	

Table 3: Yosys Gate-Level Cell Count for `regfile`

Key observations:

- Exactly **1,024 flip-flops** were inferred, confirming that all 32 registers × 32 bits are correctly mapped to sequential storage elements.
- **No latches** were inferred, confirming clean synchronous design with no unintended combinational feedback loops.
- The bypass/forwarding logic adds approximately 200 combinational cells of overhead — a negligible cost for eliminating a class of pipeline data hazards.

7 Verification Summary

Testbench	Result	Coverage
`regfile_tb.v`	**1,165 / 1,165 PASSED**	Reset state, x_0 protection, full address sweep, dual-port concurrent reads, write-through bypass, 500-step randomised stress test
`regfile_alu_tb.v`	**21 / 21 PASSED**	Register initialisation, single-cycle ALU ops (ADD, SUB, SLL, SRL, SRA, AND, OR, XOR, SLT, SLTU), multi-cycle ALU ops (MUL, DIV, REM), x_0 writeback protection, chained dependency pipeline

Table 4: Verification Results

8 Conclusion

The Register File is fully implemented, synthesised, and verified. It passes all 1,186 test vectors across the standalone and integration testbenches. The module correctly implements the RISC-V RV32I register file specification with internal write-through bypass for data hazard mitigation.

Combined with the previously completed ALU, the Meowcessor now has a functional execution datapath capable of performing all RV32IM arithmetic, logic, shift, comparison, multiplication, and division operations with register-level operand sourcing and result write-back.

The next integration target is the **Instruction Decoder (ID Stage)**, which will map 32-bit RV32IM machine code into the control signals (`rs1`, `rs2`, `rd`, `alu_op`) that drive this datapath.